1. Implement and demonstrate the FIND-S algorithm for finding the most specific hypothesis based on a given set of training data samples.

```
import matplotlib.pyplot as plt

data = [
    ['Sunny', 'Warm', 'Normal', 'Strong', 'Warm', 'Same', 'Yes'],
    ['Sunny', 'Warm', 'High', 'Strong', 'Warm', 'Same', 'Yes'],
    ['Rainy', 'Cold', 'High', 'Strong', 'Warm', 'Change', 'No'],
    ['Sunny', 'Warm', 'High', 'Strong', 'Cool', 'Change', 'Yes']
]

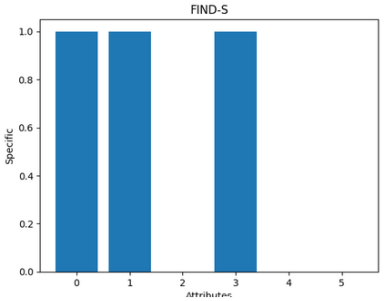
h = ['0', '0', '0', '0', '0', '0']

for row in data:
    if row[6] == 'Yes':
        for i in range(6):
            if h[i] == '0':
                h[i] = row[i]
            elif h[i] != row[i]:
                h[i] = '?'

print("Hypothesis:", h)

plt.bar(range(6), [1 if x != '?' else 0 for x in h])
plt.xlabel("Attributes")
plt.ylabel("Specific")
plt.title("FIND-S")
plt.show()
```

```
Hypothesis: ['Sunny', 'Warm', '?', 'Strong', '?', '?']
```



2. For a given set of training data examples stored in a .CSV file, implement and demonstrate the Candidate-Elimination algorithm in python to output a description of the set of all hypotheses consistent with the training examples

```
import matplotlib.pyplot as plt

data = [
    ['Sunny', 'Warm', 'Normal', 'Strong', 'Warm', 'Same', 'Yes'],
    ['Sunny', 'Warm', 'High', 'Strong', 'Warm', 'Same', 'Yes'],
    ['Rainy', 'Cold', 'High', 'Strong', 'Warm', 'Change', 'No'],
    ['Sunny', 'Warm', 'High', 'Strong', 'Cool', 'Change', 'Yes']
]

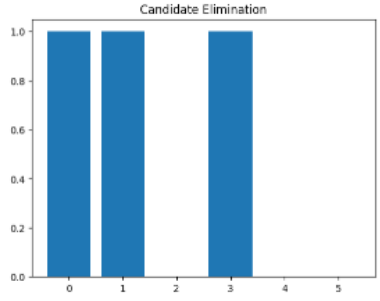
S = ['0', '0', '0', '0', '0', '0']

for row in data:
    if row[6] == 'Yes':
        for i in range(6):
            if S[i] == '0':
                S[i] = row[i]
            elif S[i] != row[i]:
                S[i] = '?'

print("Specific Boundary:", S)

plt.bar(range(6), [1 if x != '?' else 0 for x in S])
plt.title("Candidate Elimination")
plt.show()
```

```
Specific Boundary: ['Sunny', 'Warm', '?', 'Strong', '?', '?']
```



3. Demonstrate the working of the decision tree based ID3 algorithm. Use an appropriate data set for building the decision tree and apply this knowledge to classify a new sample.

```
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier, plot_tree
import matplotlib.pyplot as plt

iris = load_iris()

X = iris.data
y = iris.target

model = DecisionTreeClassifier(criterion='entropy', random_state=1)

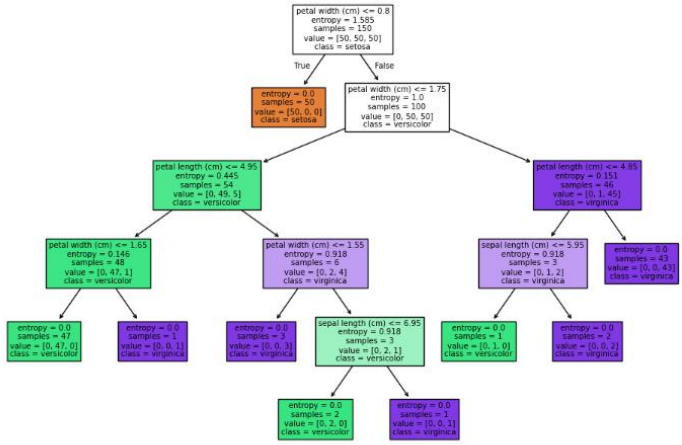
model.fit(X, y)

prediction = model.predict([[5.1, 3.5, 1.4, 0.2]])

print("Predicted Class:", iris.target_names[prediction][0])

plt.figure(figsize=(12, 8))
plot_tree(model, feature_names=iris.feature_names,
          class_names=iris.target_names, filled=True)
plt.show()
```

```
Predicted Class: setosa
```



4. Build an Artificial Neural Network by implementing the Backpropagation algorithm and test the same using appropriate data sets.

```
from sklearn.neural_network import MLPClassifier
import matplotlib.pyplot as plt

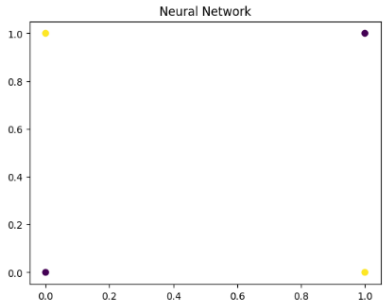
X = [[0,0],[0,1],[1,0],[1,1]]
y = [0,1,1,0]

model = MLPClassifier(hidden_layer_sizes=(2,), max_iter=1000)
model.fit(X,y)

print("Prediction:", model.predict([[1,1]]))

plt.scatter([0,0,1,1],[0,1,0,1],c=y)
plt.title("Neural Network")
plt.show()
```

```
Prediction: [1]
/usr/local/lib/python3.13/dist-packages/sklearn/neural_network/_multilayer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (1000) reached and the optimization hasn't converged yet.
  warnings.warn(
```



5. Write a program for Implementation of K-Nearest Neighbours (K-NN) in Python

```
from sklearn.neighbors import KNeighborsClassifier
import matplotlib.pyplot as plt

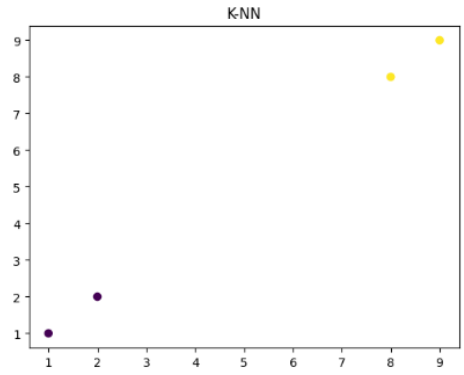
X = [[1,1],[2,2],[8,8],[9,9]]
y = [0,0,1,1]

model = KNeighborsClassifier(3)
model.fit(X,y)

print("Prediction:", model.predict([[3,3]]))

plt.scatter([1,2,8,9],[1,2,8,9],c=y)
plt.title("K-NN")
plt.show()
```

```
Prediction: [0]
```



6. Write a program to implement Naïve Bayes algorithm in python and to display the results using confusion matrix and accuracy.

```
from sklearn.naive_bayes import GaussianNB
import matplotlib.pyplot as plt

X = [[1],[2],[3],[4],[5],[6]]
y = [0,0,0,1,1,1]

model = GaussianNB()
model.fit(X,y)

p = model.predict(X)

print("Prediction:", p)
print("Accuracy:", model.score(X,y))

plt.scatter(X,y,c=p)
plt.title("Naive Bayes")
plt.show()
```

```
Prediction: [0 0 0 1 1 1]
Accuracy: 1.0
```

